

A Modified-Realizability Development for HA^ω in Lean 4: Intrinsically-Typed System T, Proof Extraction, and Zero-Axiom Verification

Ara Aslyan

August 14, 2026

Abstract

We present a complete, kernel-verified formalization of Heyting Arithmetic in all finite types (HA^ω) and Kreisel’s modified realizability in the Lean 4 interactive theorem prover. The development centers on an intrinsically typed representation of Gödel’s System T, a natural deduction proof calculus (`Deriv`) for higher-type intuitionistic arithmetic, an automated extraction functor (`extractClosed`), and a verified meta-theoretic soundness theorem (`soundnessClosed`).

A foundational discipline of this work is **strict axiom budgeting**: extracted System T realizers evaluate within Lean’s kernel with **zero non-logical axioms**, while the meta-theoretic soundness proof isolates `Classical.choice` strictly to semantic truth evaluation. We couple this core proof-theoretic engine with an operational code generator targeting Haskell and Scheme, and demonstrate the end-to-end pipeline on both discrete and continuous benchmarks: choice-free arithmetic synthesis ($n \mapsto 2n$, $n \mapsto 2^n$), rational root approximation ($\sqrt{2}$ via the Intermediate Value Theorem), certified Picard-Banach fixed-point solvers for differential equations, and symplectic orbital integrators.

1 Introduction

Program extraction via constructive logic, pioneered by Gödel [4], Kleene, and Kreisel [5], bridges formal mathematical proofs and certified computation. While mechanized proof assistants internally embody constructive type theories, formalizing the *object-level meta-theory* of arithmetic proof calculi and verifying the extraction pipeline inside the host proof assistant remains a central challenge in applied proof theory.

Existing formalizations of program extraction typically adopt one of two extremes:

1. **Shallow Embedding:** Constructive mathematical proofs are carried out directly as terms of the ambient type theory (e.g., C-CoRN [3] in Coq). This conflates the meta-logic with the object-logic, preventing fine-grained proof-theoretic inspection.
2. **External Meta-Programs:** Extraction is implemented as an unverified meta-tactic or external compiler (e.g., Minlog’s Lisp core [6]), lying outside the formal trusted computing base.

In this paper, we present a **fully certified, deeply embedded development of HA^ω in Lean 4**. Every component—from syntax, variable binding, typing, and natural deduction derivations (`Deriv`), to modified realizability (`mr`), extracted term synthesis (`extractClosed`), meta-theoretic soundness, and emission to standalone Haskell/Scheme source code—is verified inside Lean 4’s trusted kernel across 7,895 jobs.

2 Syntax, Types, and Two-Stage Substitution

The type system of HA^ω comprises base types ($\mathbb{N}$, $\mathbf{1}$) and finite type constructors:

$$\sigma, \tau ::= \text{nat} \mid \text{unit} \mid \sigma \times \tau \mid \sigma \rightarrow \tau$$

In Lean 4, terms are defined as an intrinsically well-typed family $\text{Tm} (\Gamma : \text{List Ty}) (\tau : \text{Ty})$, indexed by typing context Γ and target type τ . Variables are well-typed de Bruijn indices $\text{Var } \Gamma \tau$:

```
inductive Tm : List Ty → Ty → Type where
| var : Var Γ → Tm Γ τ
| lam : Tm (σ :: Γ) τ → Tm Γ (.arr σ τ)
| app : Tm Γ (.arr σ τ) → Tm Γ σ → Tm Γ τ
| zero : Tm Γ .nat
| succ : Tm Γ .nat → Tm Γ .nat
| recNat : Tm Γ τ → Tm Γ (.arr .nat (.arr τ τ)) → Tm Γ .nat → Tm Γ τ
| pair : Tm Γ σ → Tm Γ τ → Tm Γ (.prod σ τ)
| fst : Tm Γ (.prod σ τ) → Tm Γ σ
| snd : Tm Γ (.prod σ τ) → Tm Γ τ
| star : Tm Γ .unit
```

Variable binding uses a two-stage substitution architecture:

1. **Renaming** ($\text{Ren } \Gamma \Delta$): Pure index re-indexing without term substitution.
2. **Substitution** ($\text{Sub } \Gamma \Delta$): Mapping variables to terms, extended under binders via Sub.ext .

We verify the fundamental commutation theorem asserting that evaluation commutes with substitution:

```
theorem Tm.eval_subst {Γ : List Ty} {τ : Ty} (t : Tm Γ τ) :
  ∀ {Δ : List Ty} (s : Sub Γ Δ) (e : Env Δ),
    (t.subst s).eval e = t.eval (fun _ v ↦ (s _ v).eval e)
```

3 The Natural Deduction Proof Calculus

Derivations are formalized as an inductive family $\text{Deriv } \Delta \phi$ representing natural deduction proofs in HA^ω . The rules include:

- **Intuitionistic Propositional and Quantifier Rules:** impI , impE , andI , allI , allE , exI , exE .
- **Higher-Type Induction:**

$$\frac{\Delta \vdash \phi(0) \quad \Delta \vdash \forall n, \phi(n) \rightarrow \phi(n+1)}{\Delta \vdash \forall n, \phi(n)} \quad (\text{Deriv.ind})$$

- **Equality Axioms:** Reflexivity, symmetry, transitivity, and congruence.

4 Modified Realizability and the Axiom Budget

Kreisel's modified realizability assigns to each formula ϕ a realizer type $\tau = \text{realizerTy}(\phi)$ and a realizability predicate $r \text{ mr } \phi$:

```

def MR : (ϕ : Formula Γa) → Env Γ → (realizerTy ϕ).interp → Prop
| .eq s t, e, _ => s.eval e = t.eval e
| .and ϕψ, e, r => MR ϕ e r.1 ∧ MR ψ e r.2
| .imp ϕψ, e, r => ∀x, MR ϕ e x → MR ψ e (r x)
| .all σϕ, e, r => ∀x : σ.interp, MR ϕ (Env.cons x e) (r x)
| .ex σϕ, e, r => MR ϕ (Env.cons r.1 e) r.2

```

Theorem 4.1 (Meta-Theoretic Soundness). *For any closed derivation $D : \text{Deriv } \emptyset \phi$:*

```

theorem soundnessClosed {ϕ : Formula .nil .unit} (D : Deriv .nil ϕ) :
  MR ϕ Env.nil ((extractClosed D).eval Env.nil)

```

4.1 The Axiom Separation Table

We maintain strict separation between zero-axiom extracted program reduction and semantic soundness:

Component	Formal Role	Measured Axioms (#print axioms)
extractClosed D	Program Synthesis	None (0 axioms)
(extractClosed D).eval	In-Kernel Reduction	None (0 axioms)
doublingRealizer	Extracted Arithmetic	None (0 axioms)
termDeriv / expDoublingDeriv	Proof Derivation	[propext, Quot.sound]
soundnessClosed	Meta-Soundness	[propext, Classical.choice, Quot.sound]

Table 1: Axiom Footprint Hierarchy across Extraction Components.

5 Verification Case Studies and Benchmarks

5.1 Discrete Arithmetic Synthesis

- **Linear Doubling** ($n \mapsto 2n$): Derivation `doublingDeriv` $\vdash \forall x, \exists y, y = x + x$. Extracted realizer `doublingRealizer` evaluates to $2n$ in Lean’s kernel with **0 axioms**.
- **Exponential Doubling** ($n \mapsto 2^n$): System T primitive recursion term `expDoublingTm`. Derivation `expDoublingDeriv` evaluated with [propext, Quot.sound].

5.2 Real Analysis: Rational $\sqrt{2}$ Bisection

For the Intermediate Value Theorem existence of $\sqrt{2}$ on $[1, 2]$, the extracted term `extractedSqrt2At4` computes the 4th bisection step ($2^{-4} = 1/16$ precision):

$$\text{eval}(\text{extractedSqrt2At4}) = \frac{23}{16} = 1.4375 \quad (|1.4375^2 - 2| = 0.0664 < 0.1)$$

5.3 Picard–Banach ODE Iteration ($y' = y$)

We formalize the contraction operator $T(y)(t) = 1 + \int_0^t y(s) ds$ on $[0, 1/4]$:

- **Picard–Taylor Equivalence**: Proved identity between Picard iterates and Taylor partial sums (`picard_taylor_identity`).
- **Geometric Convergence**: Proved $\|P_n - \exp\|_\infty \leq 2 \cdot 4^{-n}$ (`exp_geometric_convergence`).

- **Kernel #guard Reduction:** Exact rational evaluation verified up to:

$$P_6(1/4) = \frac{757349}{589824} \approx 1.28402540 \quad (|P_6(1/4) - e^{1/4}| < 10^{-7})$$

5.4 Symplectic Kepler Orbit Integrator

We emit and verify the 4th-order symplectic Störmer–Verlet integrator for 2-body planetary orbits into Haskell (`ODEExtracted.hs`) and verify angular momentum conservation ($\frac{d}{dt}(\mathbf{q} \times \mathbf{p}) = \mathbf{0}$, `kepler_angular_momentum_conserved`) in Lean’s kernel.

6 Related Work

- **Formalized Proof Mining in Lean 4 (Cheval [2]):** Cheval formalizes the meta-theory of Gödel’s *Dialectica* translation for extracting majorants. Our work targets modified realizability, provides an automated program extractor, and integrates end-to-end numerical ODE solving.
- **Minlog (Schwichtenberg et al. [6, 1]):** Standalone system with external unverified Scheme extraction. Our entire pipeline is verified inside Lean 4’s trusted kernel.
- **Incone (Steinberg et al. [7]) and C-CoRN (Cruz-Filipe et al. [3]):** Shallow formalizations in Coq. Our deep embedding allows formal object-logic deduction (`Deriv`) with exact axiom measurement.

7 Conclusion

We have presented a complete, kernel-verified development of HA^ω and modified realizability in Lean 4. By maintaining a strict separation between zero-axiom extracted program reduction and choice-sound semantic validation, the framework demonstrates that proof-theoretic extraction can be rigorously integrated with real analysis, certified numerical computation, and external functional code emission.

References

- [1] Ulrich Berger and Helmut Schwichtenberg. Program extraction from classical proofs. pages 77–97, 1998.
- [2] Horațiu Cheval. Formalized proof mining in Lean 4. <https://github.com/hcheval/formalized-proof-mining>, 2023.
- [3] Luís Cruz-Filipe, Herman Geuvers, and Freek Wiedijk. C-CoRN, the constructive Coq repository at Nijmegen. *Mathematical Knowledge Management (MKM)*, 3119:88–103, 2004.
- [4] Kurt Gödel. Über eine bisher noch nicht benützte Erweiterung des finiten Standpunktes. *Dialectica*, 12(3-4):280–287, 1958.
- [5] Georg Kreisel. Interpretation of analysis by means of functionals of finite type. pages 101–128, 1959.
- [6] Helmut Schwichtenberg and Stanley S. Wainer. *Proofs and Computations*. Perspectives in Logic. Cambridge University Press, 2012.

- [7] Florian Steinberg, Laurent Théry, and Holger Thies. Computing with infinite objects in Coq: Represented spaces and type-two effectivity. In *13th International Conference on Interactive Theorem Proving (ITP 2022)*, pages 29:1–29:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.